

Viewing the virtual memory of a process

You can view the virtual memory that is being taken up by a process using a number of tools. Whenever a process runs, the set of pages required by that process is known as its working set. Obviously, the working set for a process changes continuously over its lifetime as its memory requirements also change.

The kernel constantly trims pages that have not been recently used from the working set of a process. If the pages contain modified data, then those must be written to the disk, else the kernel will allocate the pages to some other process that needs them.

You can use tools like *pmmap*, *memusage*, KDE or GNOME System Monitor to view the memory usage of a process. Figure 1 shows the truncated output from the *memusage* command for the Firefox process on my machine. Note that you need to install *glibc-utils* to use *memusage*.

You can also use the simple time command to view statistics like CPU time, system time, etc, for a process. The following snippet shows the output for time when used to check the memory status for the *man date* command:

```
[root@legacy ~]# /usr/bin/time -v man date | grep -i size
Command being timed: 'man date'
User time (seconds): 0.06
System time (seconds): 0.04
Percent of CPU this job got: 87%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00:11
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 0
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 1
Minor (reclaiming a frame) page faults: 10663
Voluntary context switches: 73
Involuntary context switches: 53
Swaps: 0
File system inputs: 206
File system outputs: 16
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Tuning TLB performance

When a process that's running needs to access a memory location, it uses a linear address. On x86 machines this linear address is a 32-bit address that can grow up to 4GB of memory, and it does not refer to the actual location. To access an actual physical memory location, linear addresses must be translated into physical addresses by

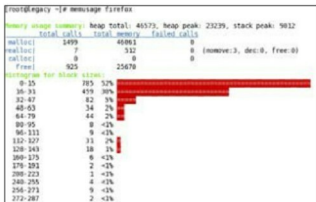


Figure 1: *memusage* output for Firefox

hardware circuits in the CPU.

Now, translating a linear address into a physical address takes time. So normally all CPUs have a small cache known as Translation Lookaside Buffer (TLB). TLB maps the physical address associated with the most recently-accessed virtual address. TLB is limited in size.

It is actually a table in the processor containing a cross-reference between virtual and real addresses of recently-referenced pages.

Tuning TLB can be very useful for applications requiring a big contiguous memory space. It is normally done at the boot time. First, check the current *hugepage* or TLB status using *cat /proc/meminfo* or *x86info* or *dmesg*. The following snippet shows the status of *hugepages* on my machine:

```
[root@legacy ~]# cat /proc/meminfo | grep Huge
HugePages_Total: 0
HugePages_Free: 0
HugePages_Rsvd: 0
HugePages_Surp: 0
Hugepagesize: 4096 kB
```

As you can see, I'm not yet using *hugepages*. So, why don't we configure the system to use the TLB or *hugepages*? Let's say I want to use 10 *hugepages* of contiguous memory.

Step 1 Configure */etc/sysctl.conf* for the number of pages you intend to use. Append *vm.nr_hugepages=10* in */etc/sysctl.conf* file and run *sysctl -p*.

Step 2 Edit */boot/grub/grub.conf* to use the *hugepages*—because this memory is allocated at the boot time—as shown below:

```
title Red Hat Enterprise Linux Server (2.6.18-02.el5)
root (hd0,0)
kernel /boot/vmlinuz-2.6.18-02.el5 ro root=LABEL=/ rhgb quiet
hugepages=10
initrd /boot/initrd-2.6.18-02.el5.img
```